

CareBridge – Architecture Decision Records

CareBridge - Online GP Consultation Platform

Author: Cretu Claudia

Program: Semester 7 Open Learning

This document records every architecture decision for CareBridge: what was decided, which options were considered, and why. It is the companion to the Software Architecture Document v2, which refers to each decision by its number.

Overview

ADR	Title
ADR-001	Record architecture decisions
ADR-002	Angular single-page application as the front-end
ADR-003	Java and Spring Boot for the back-end services
ADR-004	Three services split by business capability
ADR-005	One PostgreSQL database per service
ADR-006	Login with Google Sign-In
ADR-007	Synchronous REST between services; no message broker yet
ADR-008	Early deployment of Docker containers to a managed container platform
ADR-009	Two repositories: one for the front-end, one for the back-end services

ADR-001 Record architecture decisions

Context. The previous architecture document fixed many technologies at once without explaining why. A reader could not tell which choices were essential and which were premature, and I could not easily revisit a choice later because the reasoning was never written down.

Options considered.

Option	Pros	Cons
Keep all reasoning inside the architecture document	One place to read	Reasoning gets lost when the document is rewritten; history of changes is invisible
Lightweight ADRs, one per decision	Short, numbered, easy to reference; the history of a decision stays visible	A second document to maintain
No written decisions, rely on code and commits	No extra work	Reasons disappear; impossible to defend choices in an assessment

Decision. Every architecture decision is recorded as a short ADR in this document, using the template in the overview. The architecture document only states the result and refers to the ADR.

Consequences. Each decision can be discussed on its own, and a changed decision leaves a trail (a new ADR supersedes the old one). It also forces me to take a decision only when I can argue it, which keeps premature choices out.

ADR-002 Angular single-page application as the front-end

Context. CareBridge has three roles (patient, GP assistant, GP) with very different screens: a long validated form, a sort-able queue with drag and drop, and a booking page that may only open with a valid link. All of them run in a browser; a mobile app is out of scope.

Options considered.

Option	Pros	Cons
Angular	Complete framework: routing with route guards, reactive forms with validation, HTTP client and dependency injection built in; TypeScript by default; Angular CDK offers accessible drag and drop; commonly paired with Java back-ends in enterprise projects	Steeper learning curve and more boilerplate than lighter libraries
React	Very large ecosystem, flexible	Routing, forms and state need separate libraries that I would have to choose and justify one by one
Server-side rendering with Spring (Thymeleaf)	No separate front-end project	Harder to build an interactive drag-and-drop queue; ties the UI to one back-end service, which conflicts with a multi-service back-end

Decision. One Angular single-page application serves all three roles, with role-based route guards.

Consequences. The features CareBridge needs most (route guards for the appointment link, reactive forms for the intake, drag and drop for the queue) come from the framework itself, so fewer extra libraries need their own justification. One app for all roles keeps a single code base; the trade-off is that role separation must be enforced in the back-end as well, not only by hiding screens.

ADR-003 Java and Spring Boot for the back-end services

Context. The back-end handles sensitive health data, validation rules, and a small number of REST APIs per service. It must run in containers, be testable with automated unit tests, and support secure token validation.

Options considered.

Option	Pros	Cons
Java with Spring Boot	Mature standard for REST services; Spring Security has built-in OpenID Connect resource-server support; Spring Data JPA removes most database boilerplate; strong testing support (JUnit, MockMvc, Testcontainers); runs well in Docker	Higher memory use per service than lighter run-times; more configuration to learn
Node.js with Express or NestJS	Same language as the front-end; light and fast to start	Security and data access must be assembled from separate packages; weaker typing unless NestJS and TypeScript are used throughout
C# with ASP.NET Core	Comparable maturity to Spring	No advantage over Java for this project; would not build on the Java skills I want to deepen

Decision. Every back-end service is a Spring Boot application in Java, using Spring Web, Spring Data JPA and Spring Security.

Consequences. The same structure is used in every service (controller, service, repository), so a second and third service are faster to build than the first. Token validation is configuration rather than custom code, which reduces security risk. Memory use per service is higher, which matters for hosting cost and is a reason to keep the number of services small (ADR-004).

ADR-004 Three services split by business capability

Context. Version 1 split the back-end into four services but let them share one database and one deployment unit, so they were not independent in practice. Services only pay off when their boundaries follow real differences: parts that change for different reasons, fail in different ways, or carry different load. At the same time, learning to design and justify service boundaries is an explicit goal of this project.

Options considered.

Option	Pros	Cons
Monolith	Simplest to build, test and deploy; one database	An email-provider outage or a slow booking query affects the whole platform; offers no practice in service design
Modular monolith (modules in one deploy-able)	Clear internal boundaries with low operational cost	Modules still fail and scale together; boundaries are easy to break by accident
Four services including a self-built Identity/Auth Service (version 1)	Very fine-grained	Building login yourself is the riskiest part of a health platform; four services mean more infrastructure for one developer
Three services by business capability: Intake, Messaging, Appointment, with login handled externally	Each service maps to one step of the practice process; each can fail and scale on its own; the most security-sensitive part is not built in-house	More setup than a monolith: three pipelines, three databases, calls over the network

Decision. The back-end consists of three services, each owning one business capability: the Intake Service (submission, severity ordering, queue), the Messaging Service (GP replies and email), and the Appointment Service (links, slots, bookings). Accounts and login are delegated to an external identity provider (ADR-006).

Why these three boundaries hold up:

- *Different reasons to change:* triage and form rules (Intake), email content and provider (Messaging), practice opening hours and slot rules (Appointment).
- *Different failure impact:* if the email provider is down, patients can still submit questions and the GP assistant can still order the queue.
- *Different load:* submissions cluster when practices open in the morning, while bookings are rare; Intake can be scaled up without scaling the others.

Consequences. Each service is deploy-able, testable and scalable on its own, which is the point of the architecture. The cost is network calls between services and more deployment work, so the number of services is kept at three until a new capability clearly needs its own boundary. If the extra infrastructure turns out to slow the project down too much, merging Messaging and Appointment into one service is the fallback; that would be recorded as a new ADR.

ADR-005 One PostgreSQL database per service

Context. In version 1 all services shared one database. That couples them tightly: a table change for one service can break another, and services can read each other's data directly, so they are separate only on paper. Health data also calls for strict limits on which part of the system can see what. The hosting budget is a student budget, so running several paid database servers is not realistic.

Options considered.

Option	Pros	Cons
One shared database (version 1)	Simple joins and one connection string	Services are coupled through the schema; every service can read all health data; not a micro-services architecture
A separate database server per service	Strongest isolation	Three paid servers; more to manage than one developer needs
One database server, a separate database and user per service	Each service can only reach its own data; schema change independently; one server to pay for and manage	The server is a shared point of failure and shared capacity

For the engine itself, PostgreSQL was chosen over MySQL and MongoDB. It is open source and free, is offered as a managed service by every major cloud provider, works directly with Spring Data JPA, and its JSON column type can store the symptom checklist without forcing a rigid table for every checkbox. A document database like MongoDB was not chosen because intakes, replies and bookings have a clear, stable structure that suits a relational model.

Decision. Each service has its own PostgreSQL database with its own database user, which has rights only on that database. In the cloud the three databases run on one managed PostgreSQL server to stay within budget. No service ever queries another service's database; data from another service is only obtained through its API or from the access token.

Consequences. Services can change their own tables without coordinating with the others, and a leaked credential for one service exposes only that service's data. Queries across services (for example "all intakes with their replies") cannot be done with a join and must be combined via API calls; this is acceptable because the screens need data from one service at a time. If one service's load starts to hurt the others, its database moves to its own server without any code change, only a new connection string.

ADR-006 Login with Google Sign-In (OpenID Connect)

Context. Patients, GP assistants and GPs need accounts with different roles. Building login, password storage, password reset and account lockout by hand is the most security-critical code in a health platform and the easiest to get wrong. The platform must still know each user's role and, for patients, their email address. Google Sign-In was chosen to be investigated as the login method for all users.

Options considered.

Option	Pros	Cons
Self-built Identity/Auth Service (version 1)	Full control	Password handling, resets and lockout all built in-house; a fourth service to deploy and secure
Google Sign-In	Most patients already have an account; no passwords stored by CareBridge; free	Gives identity only, no roles, so roles must be stored elsewhere; practice staff should not log in with personal Google accounts; privacy concerns about linking health use to a Google account
Keycloak (self-hosted, open source)	Standard OpenID Connect; roles, user management and social login (including Google) built in; runs in Docker, also locally	One more container to host and keep up to date
Managed provider (e.g. Auth0, Microsoft Entra External ID)	No hosting; standard OpenID Connect; free tiers	Free-tier limits and vendor lock-in; configuration differs per provider

Proposed decision. Use Google Sign-In as the login method for patients and practice staff, and the planned implementation for CareBridge. The Angular app lets the user sign in with Google and sends the Google-issued ID token with every request. Each service validates that token with Spring Security as an OpenID Connect resource server and reads the user's Google ID and email address from it.

How roles are handled. Google only proves who someone is, not what they may do. Every user who signs in is a patient by default. GP assistants and GPs are recognised through a short allow-list of their email addresses with their role, kept in the services' configuration. Staff are asked to use a dedicated practice Google account, not a personal one.

Why Google over the other options. It removes all password handling from CareBridge, most patients can sign in without creating a new account, it is free, and it needs no extra container to host, which keeps the early deployment small. Keycloak and managed providers offer more features, such as built-in role management, but those are not needed for three roles and a handful of staff accounts.

What the spike must show. A patient and a staff member can sign in with Google from the Angular app; the Intake Service accepts a valid Google token, rejects a missing or expired one, and gives staff access only to addresses on the allow-list; sign-in works both locally and on the deployed environment.

Consequences. CareBridge stores no passwords, and login needs no extra service or container. The role allow-list is simple but must be updated by hand when staff change; if the practice grows, roles move to a proper user table or to Keycloak with Google as its login, recorded as a new ADR. Because the services only depend on the OpenID Connect standard, the provider can be swapped without rewriting service logic. Linking health use to a Google account is a privacy trade-off accepted for this learning project; a real Dutch healthcare product would likely require DigiD for patients.

ADR-007 Synchronous REST between services; no message broker yet

Context. Version 1 introduced RabbitMQ for several flows, including one event (IntakeSubmitted) that no service consumed. The current flows need only two service-to-service calls: Messaging asks Appointment for a link, and Messaging marks an intake as answered in the Intake Service. Both happen while the GP is waiting for confirmation that the reply was sent.

Options considered.

Option	Pros	Cons
Synchronous REST calls	Simple to build, test and debug; the GP gets an immediate success or error; no extra infrastructure	If the called service is down, the calling request fails and must be retried
Message broker (RabbitMQ) now	Services stay working when another is briefly down; one event can reach many	Extra container to host and monitor; asynchronous flows are harder to test and debug;

Option	Pros	Cons
	consumers	no current flow has more than one consumer

Decision. Services call each other over REST. The Angular app also calls each service over REST. A message broker is not introduced until a concrete trigger appears.

Triggers to revisit. A flow must keep working while another service is down (for example, replies being lost when the Appointment Service restarts); one event needs several consumers (for example audit logging); or retries become a real problem during testing. The reply flow in section 7.2 of the architecture document is the first candidate to move to asynchronous messaging.

Consequences. Less infrastructure and easier debugging during the semester. The Messaging Service depends on the Appointment and Intake Services being up when a reply is sent; this is handled with a clear error and a retry option for the GP.

ADR-008 Early deployment of Docker containers to a managed container platform

Context. Deployment was left until late in version 1, with Kubernetes planned and the cloud provider undecided. Problems with configuration, secrets and networking between services only show up in a real environment, and they are much cheaper to solve while the system is still small. Hosting must fit a student budget and be manageable by one person.

Options considered.

Option	Pros	Cons
Kubernetes (AKS or EKS) from the start	Industry standard; full control over scaling and rollouts	Large setup and running cost; far more than three small services need now
Managed container platform: Azure Container Apps	Runs Docker images without managing a cluster; HTTPS, scaling and rolling updates built in; can scale idle services to zero to save credit; Azure for Students gives free credit without a credit card	Less control than Kubernetes; Azure-specific configuration
Managed container platform on AWS (App Runner or ECS)	Similar features	No comparable student credit without a credit card

Option	Pros	Cons
A single virtual machine with Docker Compose	Cheapest and simplest	No managed HTTPS, scaling or rolling updates; everything done by hand

Decision. Every part of CareBridge runs as a Docker container. Locally, Docker Compose runs the whole platform. In the cloud, the containers run on Azure Container Apps, with a managed PostgreSQL server (ADR-005). The walking skeleton (increment 0) is deployed before any feature is built.

Consequences. Every increment is shown on a running environment, and deployment problems appear early. Because the unit of deployment is a standard Docker image, moving to Kubernetes later needs no change to the services themselves. Kubernetes is reconsidered if Container Apps limits scaling or rollout control. The student credit offer is confirmed with a Fontys account in increment 0; if it is not available, the single-VM option is the fallback.

ADR-009 Two repositories: one for the front-end, one for the back-end services

Context. Independent services should be built, tested and deployed independently. A change to the Appointment Service should not trigger a rebuild or redeploy of the Intake Service. At the same time, one developer works on all services and the Angular app. The front-end and the back-end use different tools: the Angular app is built with npm, the Java services with Gradle.

Options considered.

Option	Pros	Cons
One repository per service and one for the front-end	Independence is enforced by the repository boundary	Four repositories to keep in sync for one developer; shared documentation and configuration are spread out
One repository for everything, one shared build	Simplest setup	Every change rebuilds and redeploys everything, which undoes the independence of the services
One repository for everything, a folder per service and a separate pipeline per folder	One place for all code and documentation; each service still builds and deploys on its own	Mixes npm and Gradle tooling in one repository; front-end and back-end changes end up in one history
One repository for the front-	Front-end and back-end each	Discipline needed: no shared

Option	Pros	Cons
end, one for the back-end with a folder per service and a separate pipeline per folder	keep their own tooling and history; all Java services sit together with the same structure and build setup; each service still builds and deploys on its own	code between services except through their APIs

Decision. CareBridge uses two GitHub repositories. carebridge-front-end holds the Angular app. carebridge-back-end holds the three services, each in its own folder (intake-service, messaging-service, appointment-service). Each service folder has its own Gradle build and its own GitHub Actions workflow, triggered only when that folder changes. Services do not share Java modules with each other, and each deploys as its own Docker image to its own database (ADR-005).

Consequences. The front-end and back-end each have a clean history and their own pipeline. All back-end services live in one place, which makes it easy to keep them consistent and to work on a feature that touches two services. The independence of the services comes from separate builds, deployments and databases, not from separate repositories. Unlike version 1, the back-end repository no longer means one shared database or one deployment unit. If a service ever needs its own team or release rhythm, it can move to its own repository without code changes.